COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/21/26	**DATE SUBMITTED:**	10/21/26
			6
NAME:	Arididdon, Gabriel Dwyane D. Esguerra, Paula G.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 14.1 Algorithmic Complexity

1. PROCEDURE OUTPUT

Code

```
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>
#include <numeric>
#include "sortingAlgo.h"

// Parallel Merge Sort Wrapper
template <typename T>
void parallelMergeSort(T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        std::thread leftThread(mergeSortHelper<T>, arr, left, mid);
        std::thread rightThread(mergeSortHelper<T>, arr, mid + 1, right);

        leftThread.join();
        rightThread.join();

        merge(arr, left, mid, right);
    }
}

// Parallel Quick Sort Wrapper
template <typename T>
void parallelQuickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
```

```
        std::thread leftThread(quickSortHelper<T>, arr, low, pivotIndex - 1);
        std::thread rightThread(quickSortHelper<T>, arr, pivotIndex + 1, high);

        leftThread.join();
        rightThread.join();
    }
}

// Benchmark Helper using Multi-threading
void run_test(const std::string& label, std::vector<int> arr, bool isQuick) {
    auto begin = std::chrono::steady_clock::now();

    if (isQuick) {
        parallelQuickSort(arr.data(), 0, arr.size() - 1);
    } else {
        parallelMergeSort(arr.data(), 0, arr.size() - 1);
    }

    auto end = std::chrono::steady_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(end -
begin).count();

    std::cout << label << " | n = " << arr.size() << " | Time: " << elapsed << " us\n";
}

int main() {
    for (int n : {100, 1000, 10000}) {
        std::vector<int> sorted(n), reverse(n);

        std::iota(sorted.begin(), sorted.end(), 1);      // 1, 2, ..., n
        for (int i = 0; i < n; i++) reverse[i] = n - i;  // n, n-1, ..., 1

        std::cout << "===== Testing n = " << n << " =====\n";

        // Parallel Merge Sort Tests
        run_test("[Parallel Merge Sort] Best ", sorted, false);
        run_test("[Parallel Merge Sort] Worst", reverse, false);

        // Parallel Quick Sort Tests
        run_test("[Parallel Quick Sort] Best ", reverse, true);
        run_test("[Parallel Quick Sort] Worst", sorted, true);

        std::cout << "-----------------------------------------\n\n";
    }
```

```
    return 0;
}
```

```
Header

#include <cstddef>
#include <iostream>
#ifndef SORTINGALGO_H
#define SORTINGALGO_H


// ------------------ Bubble Sort --------------------
template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (int i = 0; i < arrSize; i++) {
        for (int j = i + 1; j < arrSize; j++) {
            if (arr[i] > arr[j])
                std::swap(arr[i], arr[j]);
        }
    }
}


// ------------------ Selection Sort --------------------
template<typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position;
    T smallestElem = arr[K];

    for(int j = K + 1; j < arrSize; j++) {
        if(arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;

}

template<typename T>
int selectionSort(T arr[], size_t arrSize) {
    int POS, temp, pass;
    for(int i = 0; i < arrSize; i++) {
        POS = routineSmallest(arr, i, arrSize);
```

```
        //swapping
        temp = arr[i];

        arr[i] = arr[POS];
        arr[POS] = temp;

        //for verification and counting no of passes that was done
        pass++;
    }


}

// ------------------ Insertion Sort --------------------
template<typename T>
void insertionSort(T arr[], size_t arrSize) {
    int K = 0, J, temp;

    while(K < arrSize) {
        temp = arr[K];
        J = K - 1;

        while(J >= 0 && arr[J] > temp) {
            arr[J+1] = arr[J];
            J--;
        }

        arr[J+1] = temp;
        K++;

    }
}


// ------------------ Shell Sort --------------------
template<typename T>
void shellSort(T arr[], size_t arrSize) {
    int K, J, temp;

    // outer loop: shrink the gap each pass (size/2, size/4, ..., 1)
    for (int gap = arrSize / 2; gap > 0; gap /= 2) {

        // from insertionSort
        K = gap;
        while (K < arrSize) {
```

```
            temp = arr[K];
            J = K - gap;

            // shift earlier gap-sorted elements up until the correct  location for
arr[i] is found
            while (J >= 0 && arr[J] > temp) {
                arr[J + gap] = arr[J];
                J -= gap;
            }

            arr[J + gap] = temp;
            K++;
        }
    }
}

// ------------------ Merge Sort --------------------
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;  //size of left sub-array
    int n2 = right - mid;    // size of right sub-array

    // Temporary arrays to hold sub-array data
    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // merge the temp arrays back into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // copy any remaining elements of L[]
```

```
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // copy any remaining elements of R[]
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

// Recursive helper that operates on index range [left, right]
template <typename T>
void mergeSortHelper(T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSortHelper(arr, left, mid);       // sort first half
        mergeSortHelper(arr, mid + 1, right);  // sort second half
        merge(arr, left, mid, right);          // merge sorted halves
    }
}

// for calling the function
template <typename T>
void mergeSort(T arr[], size_t arrSize) {
    if (arrSize > 0)
        mergeSortHelper(arr, 0, arrSize - 1);
}



// ------------------ Quick Sort --------------------
// Partitions arr[low..high] using arr[high] as the pivot.
// Elements less than pivot end up on its left, greater on its right.
// Returns the final index of the pivot.



template <typename T>
```

```cpp
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1; // index of last element smaller than pivot

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]); // place pivot at its correct position
    return i + 1;
}

// Recursive helper that operates on index range [low, high]
template <typename T>
void quickSortHelper(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);

        quickSortHelper(arr, low, pivotIndex - 1);   // sort left sub list
        quickSortHelper(arr, pivotIndex + 1, high);  // sort right sub list
    }
}

template <typename T>
void quickSort(T arr[], size_t arrSize) {
    if (arrSize > 0)
        quickSortHelper(arr, 0, arrSize - 1);
}




#endif
```

Output

```
===== Testing n = 100 =====
[Parallel Merge Sort] Best  | n = 100 | Time: 1230 us
[Parallel Merge Sort] Worst | n = 100 | Time: 344 us
[Parallel Quick Sort] Best  | n = 100 | Time: 330 us
[Parallel Quick Sort] Worst | n = 100 | Time: 321 us
----------------------------------------

===== Testing n = 1000 =====
[Parallel Merge Sort] Best  | n = 1000 | Time: 431 us
[Parallel Merge Sort] Worst | n = 1000 | Time: 409 us
[Parallel Quick Sort] Best  | n = 1000 | Time: 2931 us
[Parallel Quick Sort] Worst | n = 1000 | Time: 3910 us
----------------------------------------

===== Testing n = 10000 =====
[Parallel Merge Sort] Best  | n = 10000 | Time: 1556 us
[Parallel Merge Sort] Worst | n = 10000 | Time: 1527 us
[Parallel Quick Sort] Best  | n = 10000 | Time: 257136 us
[Parallel Quick Sort] Worst | n = 10000 | Time: 384010 us
----------------------------------------
```

	Best Case	Worst Case	Analysis
Algorithm 1 (Merge)	O(nlogn)	O(nlogn)	This sorting algorithm uses divide and conquer that splits the array in two halves. Which would be technically faster in sorting 100, 1000, 10000 since it is O(nlogn)
Algorithm 2 (Quick)	O(nlogn)	O(n^2)	This sorting algorithm splits the list evenly by selecting a pivot have a worst case of O(n^2) which would be slower at higher n values like 10000 especially if it did partitions unevenly.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1 (Merge)	Algorithm 2 (Quick)
Best Case	Based on the output, as the n values grows larger, the merge sort becomes faster than quick sort.	On the output of quick sort, the smaller size of 100 it executed much faster than the best case of merge sort.
Worst Case	On the worst case of merge sort, it has a faster execution time than its best case which is contrasting on the theoretical. It is maybe due to hardware and operating system or even on the thread creation overhead.	On the worst case of quick sort, the thread creation is completely warmed up, meaning in quick sort, the CPU has analyzed its part, so when it comes to the worst case, it only needs to focus on its part, which is it spawns one thread and waits for it, then repeats, it avoids hitting the OS thread scheduling bottleneck at the same time too.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.

Pseudocode

Linear Search Pseudocode Analysis

```
Function LinearSearch(array, target):
    For each index from 0 to length(array) - 1:
        If array[index] == target:
            Return index  // Target found
    Return -1  // Target not found
```

Binary Search Pseudocode Analysis

```
Function BinarySearch(sorted_array, target):
    low = 0
    high = length(sorted_array) - 1

    While low <= high:
        mid = low + (high - low) / 2
        If sorted_array[mid] == target:
            Return mid  // Target found
        Else if sorted_array[mid] < target:
            low = mid + 1  // Search in the right half
        Else:
            high = mid - 1  // Search in the left half

    Return -1  // Target not found
```

Time and Space Complexities

	Linear Search	Binary Search
Prerequisite	None (it can be unsorted or sorted data	Data should be sorted beforehand.
Worst Case Time Complexity	O(N)	O(log N)
Best Case Time Complexity	O(1)	O(1)
Space Time Complexity	O(1)	O(1)

Empirical analysis, the deliverable must be the filled-out table and your analysis.

Input Size (N)	Elapsed Time for Linear Search (Worst-Case)	Elapsed Time for Binary Search (Worst-Case)
1,000	2528 nanoseconds	31 nanoseconds
10,000	25853 nanoseconds	46 nanoseconds
100,000	251932 nanoseconds	51 nanoseconds

Analysis - Empirical Analysis

Linear Searching increases by the factor of 10, this demonstrates the O(N) time complexity, furthermore, the loop multiples by exactly 100,000 times because it uses recursion, meaning each recursion calls the memory in the computer's call stack, so if ever it runs out of memory, it triggers. Then Binary searching instead of multiplying step by step, it divides the cycle by 2, and squaring the base, this is why binary search is lesser than the operations on linear searching.

Code

```
#include <iostream>
#include <chrono>

typedef unsigned long long ull;

const int RUNS_R = 1000;
const int RUNS_B = 1000;     // Lowered to prevent terminal freeze

// O(n) Iterative Loop
ull rpower(ull x, int n) {
    ull result = 1;
    for (int i = 0; i < n; i++) {
        result *= x;
    }
    return result;
}

// O(log n) Iterative Loop
ull brpower(ull x, int n) {
    ull result = 1;
    ull base = x;
    while (n > 0) {
        if (n % 2 == 1) {
            result *= base;
        }
        base *= base;
        n /= 2;
    }
    return result;
}

void power_test(int n) {
    volatile ull result;

    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    for (int r = 0; r < RUNS_R; r++)
        result = rpower(2, n);
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
     long long time_r = std::chrono::duration_cast<std::chrono::nanoseconds>(end -
begin).count() / RUNS_R;

    begin = std::chrono::steady_clock::now();
```

```
    for (int r = 0; r < RUNS_B; r++)
        result = brpower(2, n);
    end = std::chrono::steady_clock::now();
     long long time_b = std::chrono::duration_cast<std::chrono::nanoseconds>(end -
begin).count() / RUNS_B;

    std::cout << "n = " << n << std::endl;
     std::cout << "rpower  average time taken = " << time_r << " nanoseconds" <<
std::endl;
     std::cout << "brpower average time taken = " << time_b << " nanoseconds" <<
std::endl << std::endl;
}

int main() {
    power_test(1000);
    power_test(10000);
    power_test(100000);

    return 0;
}
```

Output

```
n = 1000
rpower  average time taken = 2528 nanoseconds
brpower average time taken = 31 nanoseconds

n = 10000
rpower  average time taken = 25853 nanoseconds
brpower average time taken = 46 nanoseconds

n = 100000
rpower  average time taken = 251932 nanoseconds
brpower average time taken = 51 nanoseconds
```

Conclusion

Aridon	Esguerra
On this activity I learned how to compare the different searching and sorting algorithm by first having a theoretical analysis then having experimental one. I learned how to time and compare the two different searching and sorting algo. Although there are some discrepancies when it comes to best and worst case in experimental analysis. When it comes to linear and binary search, the binary search can handle many n values since it divides the cycles by 2 in each iteration which outputs lower operations than the linear search which has a linear growth.	In this hands-on activity, I have learned how to cope on, on how we do things, we were able to reuse our codes before and started to change them based on the instructions that we need to comply with. Furthermore, it also depends on the hardware of the computer, whether the theoretical and the actual theory works, so for example, best case is supposed to be faster than worst case, it is but the operating system need to be worked on before the coding starts, and it happens to be on the best case part, so I think to fix this, it's either make the best case to be the worst case, or have another time for the warm up before hitting the best case or worst case, just an idea. In conclusion, I have to be aware of my computer system too, and further work on my code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explaining concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used Gemini to explain hard concepts and also with some code assistance in supplementary activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.